

PHOENIX TEAVS–ADCRS JWT Authentication and Role-Based Access Control Verification Report

Project: PHOENIX

Area: Cybersecurity – TEAVS–ADCRS Integration

1. Purpose

This report documents the authentication and role-based access control testing completed for the TEAVS–ADCRS integration endpoint within the PHOENIX backend.

The main focus of this testing was the following endpoint:

POST /api/ingestion/core

The purpose was to confirm that the endpoint requires a valid JWT token and that only approved roles are able to submit data into the core model integration workflow.

The testing also included a review of the current backend implementation to confirm whether the expected RBAC control was actually being enforced.

2. Scope

The testing covered:

- JWT authentication on the core ingestion endpoint
- behaviour when no authentication token is provided
- behaviour when an invalid token is provided
- access from a valid authenticated user with an unauthorised role
- access from a valid authenticated user with an authorised role
- review of the route-level middleware
- review of the existing authentication and authorisation middleware
- implementation and verification of the required RBAC control

The purpose of this phase was specifically to verify authentication and authorisation. Input validation and payload security testing are being handled separately.

3. Endpoint Under Review

The endpoint reviewed during this phase was:

Method: POST

Endpoint: /api/ingestion/core

This endpoint is used to submit information into the core model integration workflow.

Because this endpoint can place data into the PHOENIX ingestion pipeline and RabbitMQ workflow, it should not be available to every authenticated user.

The expected access model is:

- admin – permitted
- ingestion_service – permitted
- analyst – denied
- end_user – denied
- unauthenticated users – denied

4. Initial Code Review

The first step was to review the route responsible for the core model integration.

The route was found in:

api-gateway/src/routes/ingestion.routes.ts

The original route configuration was:

```
router.post("/core", authenticate, coreModelIntegration);
```

This showed that the endpoint was protected by the authenticate middleware, meaning a valid JWT was required.

However, there was no role-based authorisation middleware attached to the route.

This meant that once a user successfully authenticated, there was no route-level check preventing roles such as analyst or end_user from reaching the core integration controller.

The controller was also reviewed to confirm whether role checking was performed internally.

The relevant controller was:

`coreModelIntegration`

The controller accepted the request body and published the data to the RabbitMQ core model integration queue.

No role verification was identified inside the controller.

This confirmed that the RBAC control was missing from the endpoint.

5. Existing Authentication and Authorisation Controls

The existing authentication middleware was reviewed in:

`api-gateway/src/middleware/auth.middleware.ts`

The middleware already contained the required functions for both authentication and authorisation.

The authenticate middleware:

- checks for a Bearer token
- verifies the JWT
- retrieves the user account
- confirms that the stored access token is still valid
- rejects missing, invalid or revoked tokens
- attaches the decoded user information to the request

The existing `authorize()` middleware performs a role check against the authenticated user's role.

If the role is not included in the allowed roles list, the middleware returns:

403 Access denied

The middleware also calls the PHOENIX security notification function when unauthorised access is detected.

This meant that the required RBAC capability already existed in the backend. The issue was that it had not been applied to the /api/ingestion/core route.

6. Security Finding

Finding ID: AUTHZ-01

Title: Missing Role-Based Access Control on Core Integration Endpoint

Affected endpoint:

POST /api/ingestion/core

Issue:

The core ingestion endpoint originally required authentication but did not enforce a specific authorised role.

The original configuration was:

```
router.post("/core", authenticate, coreModelIntegration);
```

This created a situation where any user who successfully authenticated could potentially reach the core integration controller, regardless of whether their role should be allowed to submit data into the integration pipeline.

Security Impact

The endpoint publishes data into the PHOENIX core model integration queue.

Allowing lower-privileged authenticated users to reach this functionality could result in unauthorised data entering the ingestion and analysis workflow.

For this reason, authentication by itself was not considered sufficient protection for this endpoint.

7. Remediation Implemented

The route was updated to include the existing `authorize()` middleware.

The updated implementation is:

```
router.post(
  "/core",
  authenticate,
  authorize(["admin", "ingestion_service"]),
  coreModelIntegration,
);
```

This creates two separate security checks.

The first check confirms the user has a valid authenticated session.

The second check confirms that the authenticated user has either the `admin` or `ingestion_service` role.

Other authenticated roles are rejected before the controller is executed.

8. Runtime Verification

After the RBAC change was implemented, the API Gateway was rebuilt and the endpoint was tested again.

Several authentication and authorisation scenarios were tested.

Test 1 – Request Without JWT

A POST request was submitted to:

POST `/api/ingestion/core`

without an Authorization header.

The response was:

HTTP/1.1 401 Unauthorized

Response body:

```
{  
  "status": 401,  
  "message": "No token provided"  
}
```

Result

PASS

The endpoint correctly rejects unauthenticated requests.

9. Invalid JWT Test

A request was submitted using an invalid token.

The response was:

HTTP/1.1 401 Unauthorized

Response body:

```
{  
  "status": 401,  
  "message": "Invalid token"  
}
```

Result

PASS

The authentication middleware correctly rejected the invalid JWT before the request reached the application logic.

10. Unauthorised Role Test

A controlled local test account was created with the following role:

analyst

The account successfully authenticated through the normal PHOENIX login endpoint and received a valid JWT.

The valid analyst token was then used to submit a request to:

POST /api/ingestion/core

The response was:

HTTP/1.1 403 Forbidden

Response body:

```
{  
  "status": 403,  
  "message": "Access denied"  
}
```

Result

PASS

This confirmed that a user can be successfully authenticated but still be prevented from accessing the endpoint when their assigned role is not authorised.

This is the expected behaviour for role-based access control.

11. Authorised Role Test

A second controlled local test account was created with the role:

ingestion_service

The account successfully authenticated and received a valid JWT.

The token was used to submit the following request:

POST /api/ingestion/core

The endpoint returned:

HTTP/1.1 202 Accepted

Response body:

```
{  
  "status": 202,  
  "message": "Core model integration data sent successfully"  
}
```

Result

PASS

The ingestion_service role successfully passed both authentication and authorisation.

This is important because it confirms that the new RBAC control is not simply blocking all users.

The endpoint is correctly distinguishing between an unauthorised authenticated role and an approved service role.

12. Test Results Summary

Test	Expected Result	Actual Result	Status
No JWT provided	401 Unauthorized	401 Unauthorized	PASS
Invalid JWT	401 Unauthorized	401 Unauthorized	PASS
Valid JWT – analyst role	403 Forbidden	403 Forbidden	PASS
Valid JWT – ingestion_service role	Request allowed	202 Accepted	PASS

All authentication and RBAC tests completed during this phase produced the expected result.

13. Before and After Comparison

Before Remediation

```
router.post("/core", authenticate, coreModelIntegration);
```

The endpoint required authentication but did not restrict access according to role.

After Remediation

```
router.post(
  "/core",
  authenticate,
  authorize(["admin", "ingestion_service"]),
  coreModelIntegration,
);
```

The endpoint now requires:

1. a valid authenticated user; and
2. an approved role.

This provides a stronger security boundary around the TEAVS–ADCRS integration workflow.

14. Security Event Handling

The existing authentication and authorisation middleware also integrates with the PHOENIX security notification mechanism.

Authentication and authorisation failures can trigger security events such as:

- UNAUTHORIZED_ACCESS
- INVALID_JWT

- FORBIDDEN_ACCESS

This is useful because failed authentication and RBAC attempts are not only blocked at the API layer but can also be passed into the application's security monitoring workflow.

Further verification of the notification behaviour can be completed as part of later security testing if required.

15. Additional Observation

During the authorised ingestion_service test, an empty JSON payload:

```
{}
```

was accepted by the endpoint and resulted in:

202 Accepted

This was outside the main scope of the RBAC test, but it indicates that request-body validation should be reviewed separately.

The endpoint should confirm that required TEAVS–ADCRS fields are present and valid before a message is accepted into the integration queue.

This will be investigated during the next testing phase covering input validation and error handling.

16. Security Control Assessment

Security Control	Assessment
JWT required	Verified
Missing token blocked	Verified
Invalid token blocked	Verified
Role-based access control	Implemented and verified

Analyst access blocked	Verified
ingestion_service access allowed	Verified
Security notification hooks	Present in middleware
Input validation	Requires further testing

17. Technical Contribution

During this testing phase, the existing TEAVS–ADCRS security boundary was reviewed rather than assuming that the intended security controls were already operating correctly.

The review identified that authentication had been implemented, but role-based authorisation was missing from the core integration endpoint.

The existing PHOENIX authorisation middleware was then applied to the route so that only admin and ingestion_service roles could access the endpoint.

The change was rebuilt and verified using controlled test accounts representing both an authorised and unauthorised role.

This changed the endpoint from an authentication-only control to an authenticated and role-restricted integration endpoint.

18. Evidence Collected

Evidence collected during this phase includes:

EVID-01 – Original /core route using authentication only

```

* status: req.status, message: req.cookies.provider ?
*
* (base) abdulrehman@Abduls-Air backend % sed -n '318,380p' api-gateway/src/routes/ingestion.routes.ts
*
*   type: number
*   example: 1
*   description: Severity level (1 = low, 5 = critical).
*   hazard_timestamp:
*     type: string
*     format: date-time
*     example: "2026-05-02T08:00:00Z"
*     description: Timestamp when the hazard was detected.
*   hazard_location:
*     type: string
*     example: "VIC"
*     description: Location code (e.g., state or region).
*   hazard_status:
*     type: string
*     example: "active"
*     description: Status of the hazard (active, resolved, etc.).
*   alert_level:
*     type: string
*     example: "critical"
*     description: Alert level (info, warning, critical).
*   source:
*     type: string
*     example: "OpenPhish"
*     description: Source of the intelligence.
*
*   responses:
*     202:
*       description: Core model integration request accepted
*       content:
*         application/json:
*           schema:
*             type: object
*             properties:
*               status:
*                 type: integer
*                 example: 202
*             message:
*               type: string
*               example: "Core model integration data sent successfully"
*
*     400:
*       description: Invalid request payload
*     401:
*       description: Unauthorized
*     500:
*       description: Internal server error
*
* router.post("/core", authenticate, coreModelIntegration);
*
* export default router;

```

EVID-02 – Existing authorize() middleware implementation

```

* (base) abdulrehman@Abduls-Air backend % grep -n -A 40 'export const authorize' api-gateway/src/middleware/authMiddleware.ts
*
* 85- export const authorize = (roles: string[]) => {
* 86-   return (req: Request, res: Response, next: NextFunction) => {
* 87-     const user = (req as any).user;
* 88-
* 89-     if (!user || !roles.includes(user.role)) {
* 90-       sendSecurityNotification(
* 91-         "HIGR",
* 92-         "User with role '{user?.role ?? "unknown"}' attempted to access a restricted resource.",
* 93-         req.originalUrl,
* 94-         req.method,
* 95-         req.ip,
* 96-         user?.user_id,
* 97-       );
* 98-       return res.status(HttpStatusCodes.HTTP_STATUS_FORBIDDEN).json({
* 99-         status: HttpStatusCodes.HTTP_STATUS_FORBIDDEN,
* 100-         message: "Access denied",
* 101-       });
* 102-     }
* 103-     next();
* 104-   };
* 105- };
* 106-
* 107- export const authorizeSelfForRoles = (
* 108-   roles: string[],
* 109-   paramName = "userId",
* 110- ) => {
* 111-   return (req: Request, res: Response, next: NextFunction) => {
* 112-     const user = (req as any).user;
* 113-
* 114-     const requestedUserId = req.params[paramName];
* 115-
* 116-     if (!user) {
* 117-       sendSecurityNotification(
* 118-         "HIGR",
* 119-         "Unauthenticated request attempted to access a protected user resource.",
* 120-         req.originalUrl,
* 121-         req.method,
* 122-         req.ip,
* 123-       );
* 124-     }
* 125-
* 126-     return res.status(HttpStatusCodes.HTTP_STATUS_UNAUTHORIZED).json({
* 127-       status: HttpStatusCodes.HTTP_STATUS_UNAUTHORIZED,
* 128-       message: "Unauthorized",
* 129-     });
* 130-   };
* 131-
* 132-   if (roles.includes(user.role) || user.user_id === requestedUserId) {
* 133-     return next();
* 134-   }
* 135-
* 136-   sendSecurityNotification(
* 137-     "HIGR",
* 138-     "User attempted to access another user's protected resource.",
* 139-     req.originalUrl,
* 140-     req.method,
* 141-     req.ip,
* 142-     user?.user_id,
* 143-   );

```

EVID-03 – Updated route with admin and ingestion_service RBAC

```

• (base) abdulrehman@Abduls-Air backend % tail -20 api-gateway/src/routes/ingestion.routes.ts
*
*       type: integer
*       example: 202
*
*       message:
*       type: string
*       example: "Core model integration data sent successfully"
*
*   400:
*     description: Invalid request payload
*
*   401:
*     description: Unauthorized
*
*   500:
*     description: Internal server error
*
*/
router.post(
  "/core",
  authenticate,
  authorize(["admin", "ingestion_service"]),
  coreModelIntegration,
);

export default router;

```

EVID-04 – 401 response for missing token

```

• (base) abdulrehman@Abduls-Air backend % curl -i -X POST http://localhost:3001/api/ingestion/core \
  -H "Content-Type: application/json" \
  -d '{}'
HTTP/1.1 401 Unauthorized
X-Powered-By: Express
Access-Control-Allow-Origin: *
Content-Type: application/json; charset=utf-8
Content-Length: 44
ETag: W/"2c-FtsHYt+khUOV9W9+yz/wLxZoogE"
Date: Sat, 26 Sep 2026 21:23:27 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"status":401,"message":"No token provided"}
• (base) abdulrehman@Abduls-Air backend %

```

EVID-05 – 401 response for invalid JWT

```

• (base) abdulrehman@Abduls-Air backend % export ANALYST_TOKEN='invalid-token-test'
• (base) abdulrehman@Abduls-Air backend % curl -i -X POST http://localhost:3001/api/ingestion/core \
  -H "Authorization: Bearer $ANALYST_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{}'
HTTP/1.1 401 Unauthorized
X-Powered-By: Express
Access-Control-Allow-Origin: *
Content-Type: application/json; charset=utf-8
Content-Length: 40
ETag: W/"28-u8D1EQynCil1jDMEiBgBwzgtezM"
Date: Sat, 26 Sep 2026 21:24:32 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"status":401,"message":"Invalid token"}

```

EVID-06 – Controlled analyst test account

```

# export RBAC_TEST_PASSWORD='PhoenixTemp2026IX'
# node -e "const bcrypt=require('bcrypt'); const {UserAccount}=require('/app/dist/libs/common/src/databases/models/user-account.model.js'); const {sequelize}=require('/app/dist/libs/common/src/databases/sequelize-connection-handler.js'); (async()=>{const username='rbac_test_analyst'; let user=await UserAccount.findOne({where:{username}}); if(!user){const password_hashed=await bcrypt.hash(process.env.RBAC_TEST_PASSWORD,10); user=await UserAccount.create({username,password_hashed,role:'analyst'}); console.log('Created RBAC test user');}else{console.log('RBAC test user already exists'); console.log('username:',user.username); console.log('role:',user.role); console.log('user_id:',user.user_id); await sequelize.close();}}()).catch(async e=>{console.error(e); try{await sequelize.close();}catch{} process.exit(1)}});"
# injected env (0) from .env // tip: * secrets for agents [www.dotenv.com]
# injected env (0) from .env // tip: * custom filepath { path: '/custom/path/.env' }
Created RBAC test user
username: rbac_test_analyst
role: analyst
user_id: ec00e3d9-268d-4806-a5fb-abc3fb08b339
# exit
(base) abdulrehman@Abduls-Air backend % curl -s -X POST http://localhost:3001/api/users/auth/login \

```

EVID-07 – 403 response using valid analyst JWT

```

(base) abdulrehman@Abduls-Air backend % export ANALYST_TOKEN=$(curl -s -X POST http://localhost:3001/api/users/auth/login \
-H "Content-Type: application/json" \
-d '{
  "username": "rbac_test_analyst",
  "password": "PhoenixTemp2026!X"
}') | python3 -c 'import sys,json; print(json.load(sys.stdin)["access_token"])'
(base) abdulrehman@Abduls-Air backend % curl -i -X POST http://localhost:3001/api/ingestion/core \
-H "Authorization: Bearer $ANALYST_TOKEN" \
-H "Content-Type: application/json" \
-d '{}'
HTTP/1.1 403 Forbidden
X-Powered-By: Express
Access-Control-Allow-Origin: *
Content-Type: application/json; charset=utf-8
Content-Length: 40
ETag: W/"28-LfLQkh0Y/RvjKgkKudXfL07nF5s"
Date: Sat, 26 Sep 2026 21:28:54 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"status":403,"message":"Access denied"}
(base) abdulrehman@Abduls-Air backend %

```

EVID-08 – Controlled ingestion service test account

```

(base) abdulrehman@Abduls-Air backend % docker compose exec user:service sh
# node -e 'const {UserAccount} = require("/app/dist/libs/common/src/databases/sequelize/models/user-account.model.js"); const {sequelize} = require("/app/dist/libs/common/src/databases/sequelize-connection-handler.js"); (async) => {const user = await UserAccount.findOne({where: {username: "rbac_test_ingestion"}}); console.log("username:", user.username); console.log("role:", user.role); console.log("user_id:", user.user_id); await sequelize.close();}}().catch(async err => console.error(err)); try{await sequelize.close();}catch{} process.exit(1);}'
# injected env (0) from .env // tip: * override existing ( override: true )
# injected env (0) from .env // tip: * suppress logs ( quiet: true )
username: rbac_test_ingestion
role: ingestion_service
user_id: cb0d027b-9dee-43d7-8f53-ba2d529f995f
# exit
(base) abdulrehman@Abduls-Air backend %

```

EVID-09 – 202 response using valid ingestion service JWT

```

(base) abdulrehman@Abduls-Air backend % export INGESTION_TOKEN=$(curl -s -X POST http://localhost:3001/api/users/auth/login \
-H "Content-Type: application/json" \
-d '{
  "username": "rbac_test_ingestion",
  "password": "PhoenixTemp2026!Y"
}') | python3 -c 'import sys,json; print(json.load(sys.stdin)["access_token"])'
(base) abdulrehman@Abduls-Air backend % echo ${INGESTION_TOKEN}
273
(base) abdulrehman@Abduls-Air backend % curl -i -X POST http://localhost:3001/api/ingestion/core \
-H "Authorization: Bearer $INGESTION_TOKEN" \
-H "Content-Type: application/json" \
-d '{}'
HTTP/1.1 202 Accepted
X-Powered-By: Express
Access-Control-Allow-Origin: *
Content-Type: application/json; charset=utf-8
Content-Length: 72
ETag: W/"48-RWthroSn8m+IA+xa+VR31/NSCFLM"
Date: Sat, 26 Sep 2026 21:08:05 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"status":202,"message":"Core model integration data sent successfully"}

```

JWT values and test passwords should not be included in screenshots or final project documentation.

19. Overall Assessment

The authentication control on the TEAVS–ADCRS core ingestion endpoint was functioning correctly.

However, the initial review identified a missing role-based authorisation control.

This issue was corrected by applying the existing PHOENIX authorize() middleware and restricting access to:

admin

and

ingestion_service

Runtime testing confirmed that:

- unauthenticated users are rejected;
- invalid JWTs are rejected;
- authenticated analysts are denied access;
- authorised ingestion service accounts are allowed through.

Based on the testing completed in this phase, the JWT and RBAC controls for /api/ingestion/core are now operating as expected.

The next security testing phase should focus on input validation, malformed payloads and error handling because the endpoint currently accepted an empty JSON object and returned 202 Accepted.

20. Conclusion

This phase confirmed both the existing JWT authentication behaviour and the newly enforced RBAC control for the TEAVS–ADCRS integration endpoint.

The main security issue identified during the review was the absence of role-based authorisation on the core ingestion route.

The issue was corrected and verified through runtime testing using both authorised and unauthorised roles.

The endpoint now provides a clearer security boundary between general authenticated users and the accounts that are actually permitted to submit information into the core integration workflow.